Find:

b	c	a	e	d

Text:

a	x	e	b	c	a	e	c	b	c	f	b	c	a	e	d	e	c	a	g	b	a	f	g	b	c	a	e	d	f

Introduction to Algorithms

Institute of
Algorithms and Theory

Yannic Maus

Algorithms:

Step-by-step instructions for solving (computational) problems.

Goal: Find the most efficient solution methods possible.

Data Structures:

Methods for storing and organizing data.

Goal: Enable efficient access to and modification of data.

First central question of this course: *What does it mean to be efficient?*

Algorithms, Data Structures, and efficiency go hand-in-hand and cannot be separated.

"Chicken or egg, which came first?" *We first focus on algorithms and efficiency …*

Disclaimer

The lecture is independent of the programming language.
The learning objective is the individual algorithms, data structures, design techniques, and their analysis, and not their implementation.

Algorithms

!Algorithms make the world go round!

Essentially no part of our life would look the same without algorithms.

E.g., Transport industry, food industry, all engineering disciplines, entertainment industry, universities, …

Algorithm design ≠ programming

(classically "programmer" is also an "Ausbildungsberuf")

Goals:

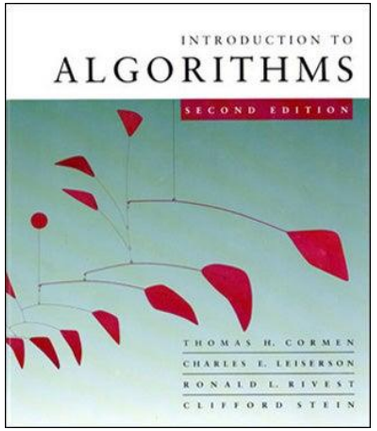
- Formal understanding of algorithms
- Huge toolbox (data structures and algorithms)
- Resource analysis: What is the efficiency of an algorithm?
- Design principles

Timely?

Do we need to re-think algorithms in the future?

Literature:

- **CLRS** – *Introduction to Algorithms* (3rd edition, 2009, 1st edition, 1990)
- **Knuth** – *The Art of Computer Programming* (Vol. 1, 1968)
- **Tarjan** – *Data Structures and Network Algorithms* (1983)



Pro Tip: Read these books!

Significant Impact in the Past:

- PageRank algorithm made Google successful.
- Almost every computer interaction involves algorithms taught in our classes running in the background.

Timely?

Still Relevant?

- **Yes!**
- **Machine Learning**: Classic algorithms, such as decision trees or k-nearest neighbors, run in the background of applications like recommendations on streaming platforms (e.g., Netflix, Spotify).
- **[SPAA 2018 Best Paper]**: *Theoretically Efficient* ***Parallel*** *Graph Algorithms Can Be Fast and Scalable* by Laxman Dhulipala, Guy E. Blelloch, and Julian Shun—now in daily use at Google.

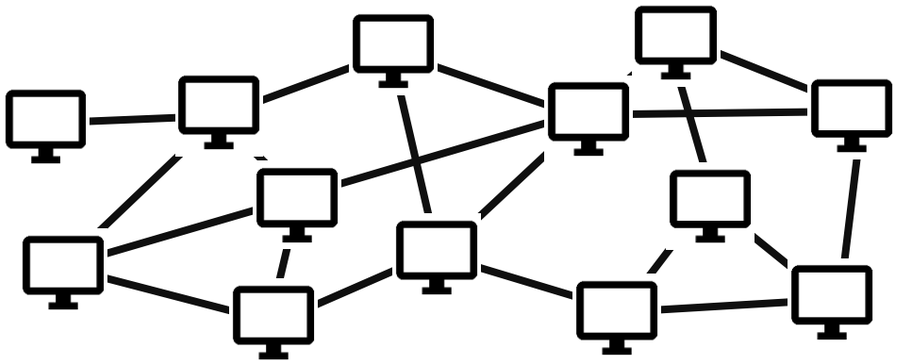
FAANG/MAMAA Interviews:

- Focus on **Algorithm Design + Implementation**

Outlook: Modern Models of Computation

Parallel Algorithms for Data centers

Outlook: Modern Models of Computation



Algorithms for large networks

This course: Sequential Computing

Computational Problem

Computational problem:

- **Input:** A set of valid instances (e.g., numbers, strings, graphs).
- **Output:** A set of possible outputs.
- **Input-Output Relation:** A relation $\subseteq \mathrm{Inputs} \times \mathrm{Outputs}$ between inputs and outputs that states which outputs are correct for the respective input. Each valid input instance needs to be related to at least one correct output.

Instance of a computation problem: concrete input

Example: The "computational problem" of **searching in an array**

- **Input:** Sequence $A = (a_1, a_2, a_3, \dots, a_n)$ of n numbers and a number x
- **Input-Output Relation:** For input (A, x) all indices $i \in \{1, \dots, n\}$ with $a_i = x$ are correct outputs. If no such index exists the correct output is -1.

Computational Problem: Example Searching

Example: The "computational problem" of **searching in an array**

- **Input:** Sequence $A = (a_1, a_2, a_3, \dots, a_n)$ of n numbers and a number x
- **Input-Output Relation:** For input (A, x) all indices $i \in \{1, \dots, n\}$ with $a_i = x$ are correct outputs. If no such index exists the correct output is -1.

Instances+Input-Output Relation:

- **Array:** A=[3, 1, 4, 1, 5, 9, 2, 6, 5, 3], **x = 5** (appears twice in the array)
  I/O-relation contains: ((A,x),4), ((A,x),8)
- **Array:** A=[8, 2, 7, 3, 9, 4, 1, 6, 5, 10], **x = 10** (last element in the array)
  I/O-relation contains: ((A,x),9)
- **Array:** A=[2, 3, 5, 7, 11, 13, 17, 19, 23, 29], **x = 15** (not in the array)
  I/O-relation contains: ((A,x),-1)
- **Array:** A=[7, 4, 9, 2, 6, 3, 8, 1, 5, 10], **x = 7** (first element in the array)
  I/O-relation contains: ((A,x),0)

First Algorithm: Linear Search

Idea: Go through the array from start to end, return index i if A[i]==x

```
LinearSearch(A, x):
    for i = 0 to n - 1 do
        if A[i] == x then
            return i  // x found at index i
    return -1  // x not found
```

Analyzing the algorithm?

- Part I: Correctness
- Part II: Resource consumption (runtime and memory/space usage).

Correctness of an Algorithm

Correctness of an Algorithm:
An algorithm is **correct** if, for every valid input, it produces a correct output after a finite number of steps.

Correctness of the Linear Search Algorithm:

Always terminates:

- Each iteration of the FOR loop takes finite time.
- At most **n** iterations.

Always produces the correct output using the loop invariant:

- After the **i-th** iteration of the FOR loop, **x** is not present in **A[0..i-1]**.
  - (Base case: **i = 0**)
- **Output -1:** Only after the loop → **x** is not in **A[0..n-1]**.
- **Output j, 0 ≤ j ≤ n-1:** Only if **A[j] = x**.

Correctness with Hoare Logic

Formally use Hoare logic to show the correctness of a program!

We will stick to the basics in this lecture and Hoare Logic is not part of this course

Runtime Analysis is difficult

Consider Algorithm A, Algorithm B, and some fixed input x.

Result:

- Algorithm A needs 10 seconds to produce an output
- Algorithm B needs 15 seconds to produce an output

Is Algorithm A faster? It is, on input x on the computer where we ran it!

But there are many possible other inputs and other computers.

Example: two versions of LinearSearch

```
for i=0 to n-1:
    if A[i] == x: return i
return -1
```

```
for i=n-1 to 0:
    if A[i] == x: return i
return -1
```

Which one is better? Depends on the instance …

Runtime as a function of input size

Simplification I:

The runtime of an algorithm is not a single number but a function T: $\mathbb{N} \to \mathbb{N}$.
It represents the runtime of the algorithm for each **input size**.

- **Worst-case runtime:** The runtime for an input length $\boldsymbol{n}$ is the **worst runtime** among **all** inputs of length $\boldsymbol{n}$.

  Example: LinearSearch on instance A=[2, 3, 5, 7, 11, 13, 17, 19, 23, 29], **x = 15**

  (x=15 not contained, need to go through the whole array)

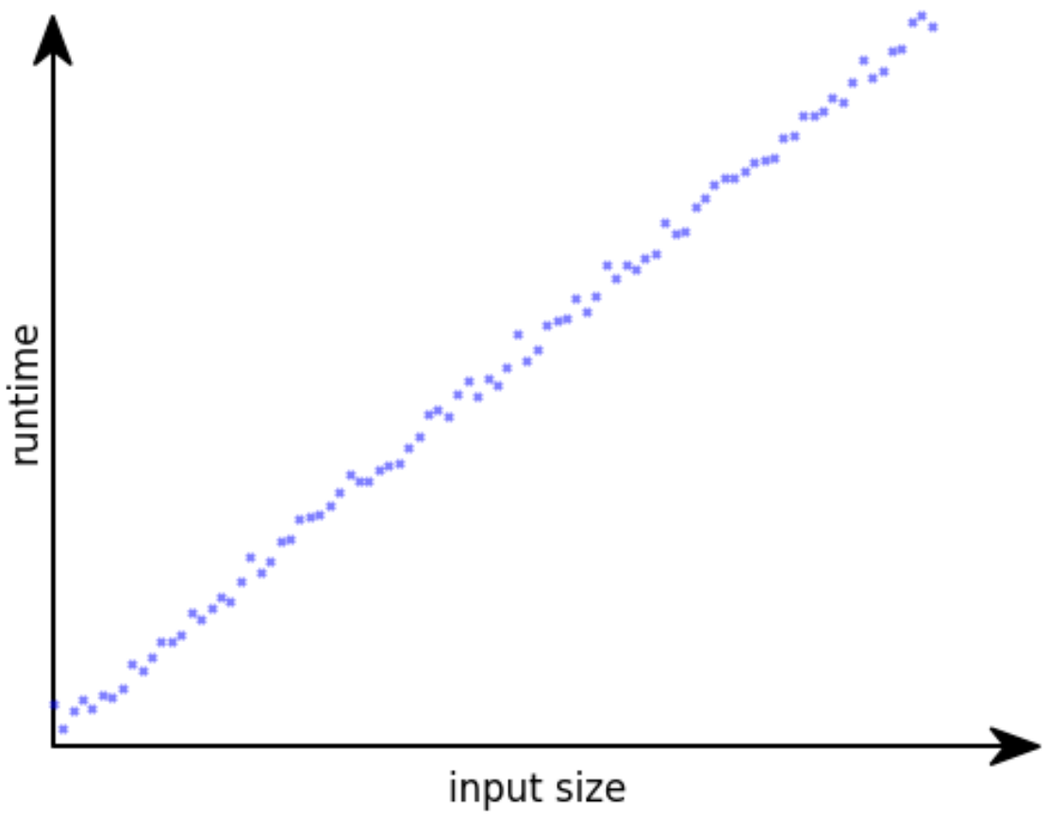
- **Best-case runtime:** The runtime for an input length $\boldsymbol{n}$ is the **best runtime** among **all** inputs of length $\boldsymbol{n}$.

  Example: LinearSearch on instance A=[7, 4, 9, 2, 6, 3, 8, 1, 5, 10], **x = 7**

  (x=7 at first position, done after first iteration of for-loop)

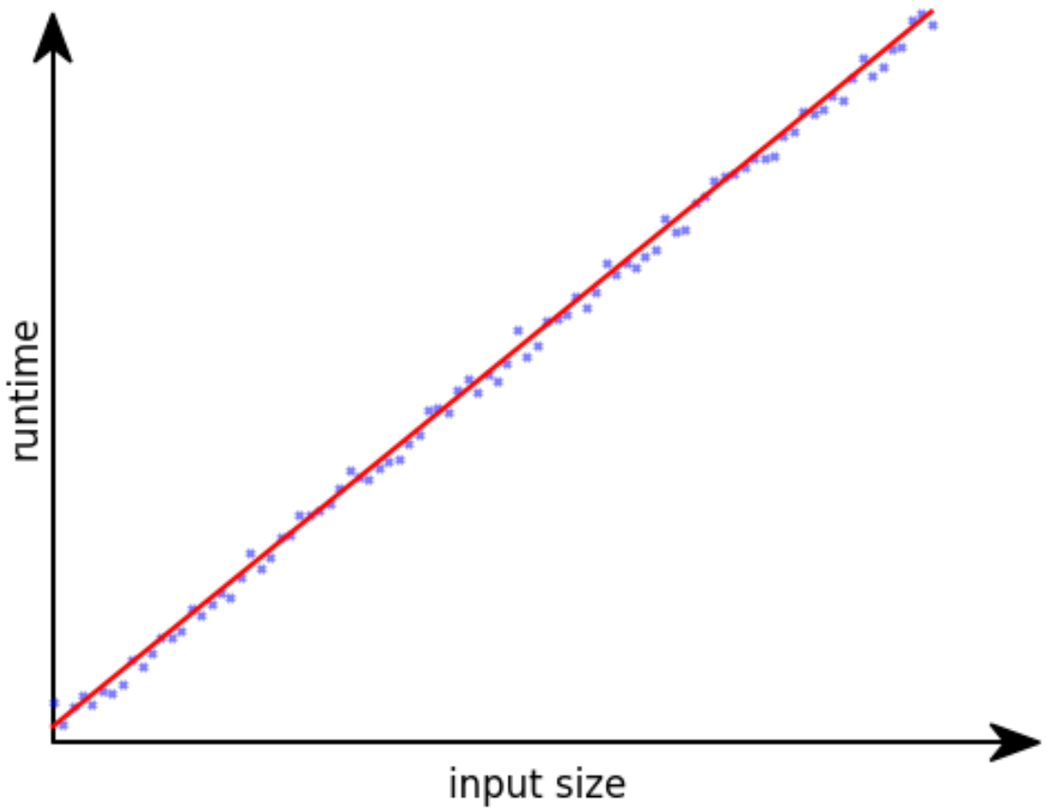
Linear Search: Runtime measurements

Runtime vs. Input size (searching for a random element in the array)



Linear Search: Runtime measurements

Runtime vs. Input size (searching for a random element in the array)



Linear Search: Runtime I

```
LinearSearch(A, x):
    for i = 0 to n - 1 do
        if A[i] == x then
            return i
return -1
```

Assumptions

c_1 //runtime for initializing/incrementing i

c_2 //runtime for executing comparison

c_3 //runtime to return memory cell

c_4 //runtime to return -1

Linear Search: Runtime II

Algorithm: Linear Search

The runtime depends on whether and where *x* is found:

Best case: A[0] = x

$$T(n) = c_1 + c_2 + c_3$$

$\Rightarrow$ Constant in n

Worst case: A[n-1] = x or x ∉ A

$$T(n) = n \cdot (c_1 + c_2) + c_4$$

$\Rightarrow$ Linear in n

Average case/Expected Case:

- Difficult topic: Average among which cases? Which guarantee on the input? How to weigh which input?
- To be discussed later

Binary Search

Searching in Sorted Array

Input: Array **A** with a **sorted sequence** **of length n**, number **x**
Ouput: given implicitly

I/O relation: (A,x,i) where **A[i] = x**, or **(A,x,−1)** if **x ∉ A**

BinarySearch

Idea:

Compare **x** with the (approximate) middle element of **A**.

If they are not equal, continue searching in only one half of the array.

Binary Search Pseudocode

```
BinarySearch(A, x):
    left ← 0 right ← n - 1
     while left ≤ right do
        mid ← ⌊(left + right)/ 2⌋  // Compute middle index
        if A[mid] == x then: return mid  // x found at index mid
        else if A[mid] < x then
            left ← mid + 1  // Search in right half
        else
            right ← mid - 1  // Search in left half
     return -1  // x not found
```

Binary Search: Correctness

Invariant: If $x \in A$, then $x \in A[\text{left} \,..\, \text{right}]$ during the execution

Induction base: Holds initially.

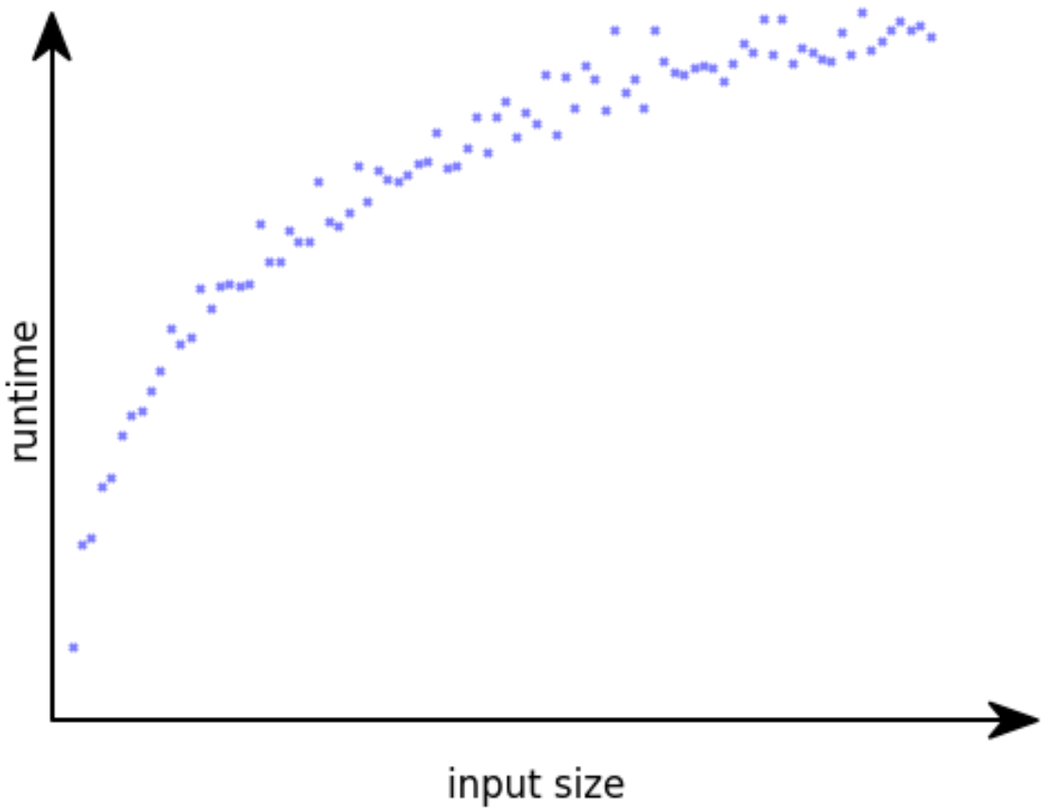
Induction over iterations:

- Case $x = A[mid]$: x found ⇒ finished, output, done, no more iterations.
  ⇒ Invariant holds (contains the case left==right)
- Case $x < A[mid]$: Because A is sorted, $x < A[i], \forall\, i \geq mid$
  ⇒ Invariant holds when restricting to $A[start \,..\, mid - 1]$
- Case $x > A[mid]$: $x > A[i], \forall\, i \leq mid$
  ⇒ Invariant holds when restricting to $A[mid + 1 \,..\, end]$

Termination: *search space is strictly smaller after each iteration*

$\text{right} - \text{left}$ decreases in each iteration; termination when finding x, or when $\text{right} - \text{left} \leq 0$.

Binary Search: Runtime



Only logarithmic growth?

Binary Search: Runtime

Assumption: Each loop iteration is constant $\leq c$

Best Case: $x = A\left[\left\lfloor\frac{n+1}{2}\right\rfloor\right]$

$T(n) \leq c$ (constant in n)

Worst Case: $x \notin A$

In each loop iteration, the search range is halved

Termination occurs at latest when the search range = 1 element

$\Rightarrow$ At most $k + 1$ loop iterations where

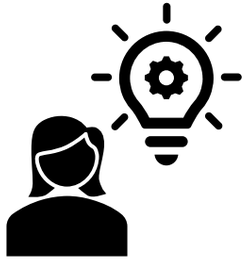
$$2^{k-1} < n \leq 2^k \Leftrightarrow \log_2(n) \leq k < \log_2(n) + 1$$

$T(n) \leq c \cdot (\lfloor \log_2(n) \rfloor + 1)$ (logarithmic in n)

BinarySearch-Concept is Helpful for the fun times in life

Game:

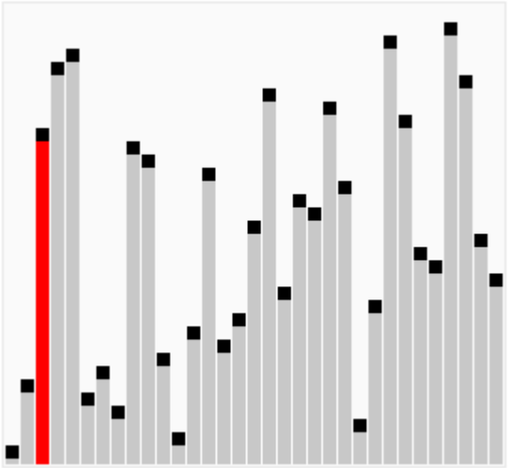
I met a person and you will never guess whome!



I'll guess it using binary search

Answers are not structured in a sorted array, but using binary search one can roughly half the search space with each yes/no question

Sorting



Computational Problem Sorting

Input: Sequence of numbers $A = \langle a_1, a_2, \ldots, a_n \rangle$

~~Output: Set of all sorted array (implicitly given)~~

Output ~~(I/O relation)~~**:** Permutation $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of A with $a'_1 \leq a'_2 \leq \ldots \leq a'_n$.

Sorting Playing Cards

How do you sort playing cards?

Algorithm?

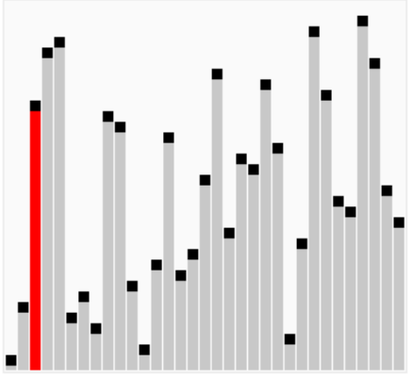
Selection Sort

Idea:

- Find smallest element and put it to the beginning,
- Find smallest element under the remaining elements,
- Etc.

More precisely:

- Find smallest element, exchange it with the element at the first position,
- Find smallest element in the rest, exchange it with the element at the second position
- Etc



This animation is different, how?

Selection Sort: Pseudocode

```
SelectionSort(A):
for i = 0 to n - 2 do
    minIdx = i
    for j = i + 1 to n - 1 do
        if A[j] < A[minIdx] then
            minIdx = j
    swap(A[i], A[minIdx])
```

Termination?
Yes, finitely many iterations of the loop

Correctness

Properties after loop iteration i:

a) A contains the same values as at the beginning
b) A[0..i] is correctly sorted
c) "Values in A[0..i]" ≤ "Values in A[i+1..n-1]"

Selection Sort: Correctness

Properties after loop iteration i:

a) A contains the same values as at the beginning
b) A[0..i] is correctly sorted
c) "Values in A[0..i]" ≤ "Values in A[i+1..n-1]"

Proof by mathematical induction:

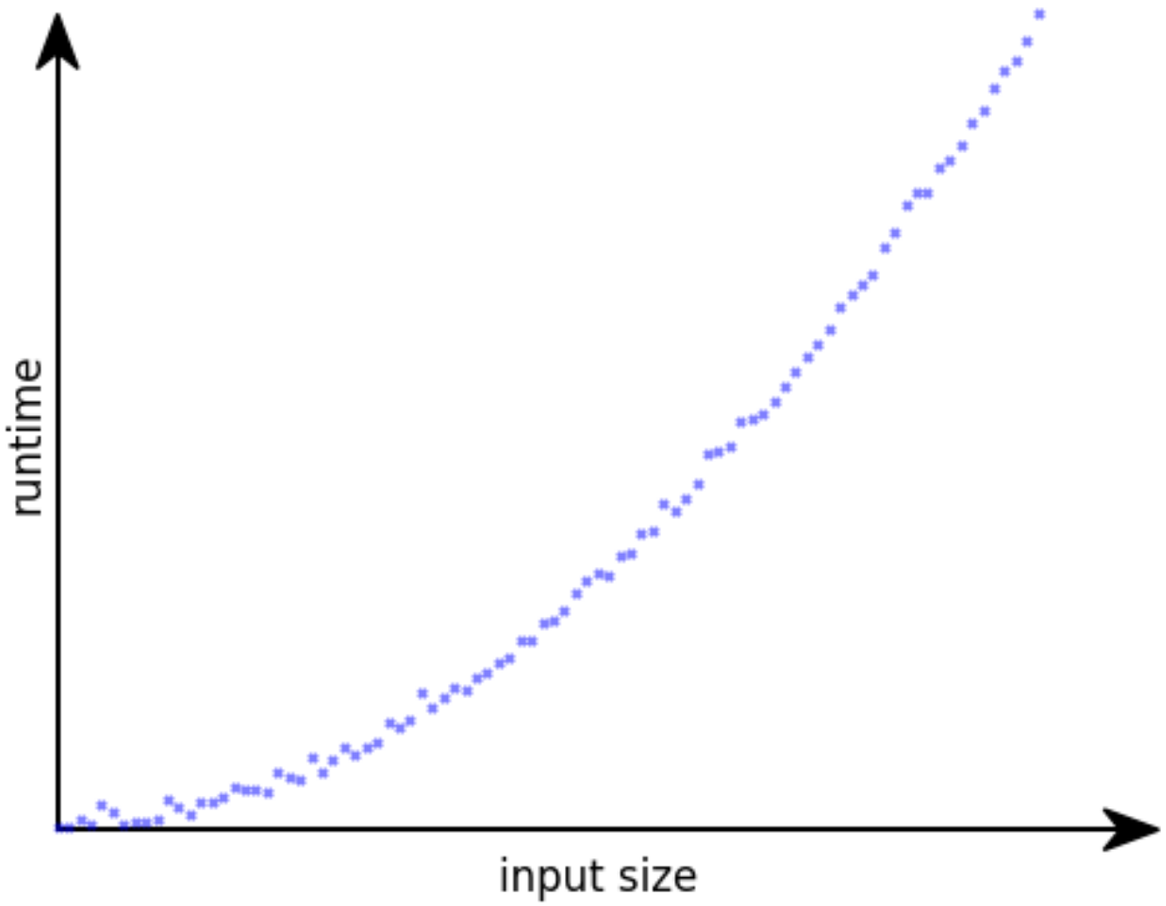
Induction Base (i=0): holds trivially (after the first iteration)

Induction hypothesis: a)-c) hold after iteration i.

Inductive step (i→i+1):

a) follows by IH a) because only swaps are made
b) Values in A[0..i] are not changed during loop iteration i+1 and correctly sorted by IH b). After iteration i+1 item A[i+1] will be an element of A[i+1,..n-1] from before the iteration, so b) after iteration i+1 follows from IH c) .
c) follows because A[i+1] will contain the smallest element in A[i+1..n-1]

Selection Sort: Runtime



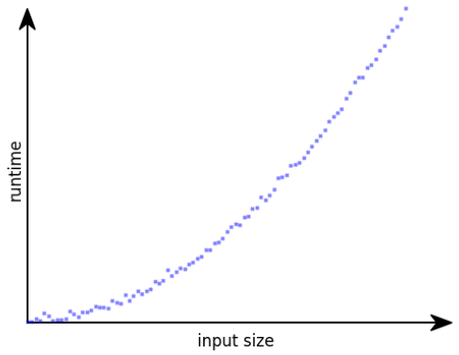
Selection Sort: Runtime

It seems to become disproportionately slower as the array size increases.

The time appears to grow roughly quadratically with the size of the array.

- A double-sized array → 4 times longer runtime
- A triple-sized array → 9 times longer runtime
- …

We will see that the runtime is indeed quadratic and how to analyze and express this formally.



First, let's think about other ways to sort...

Insertion Sort

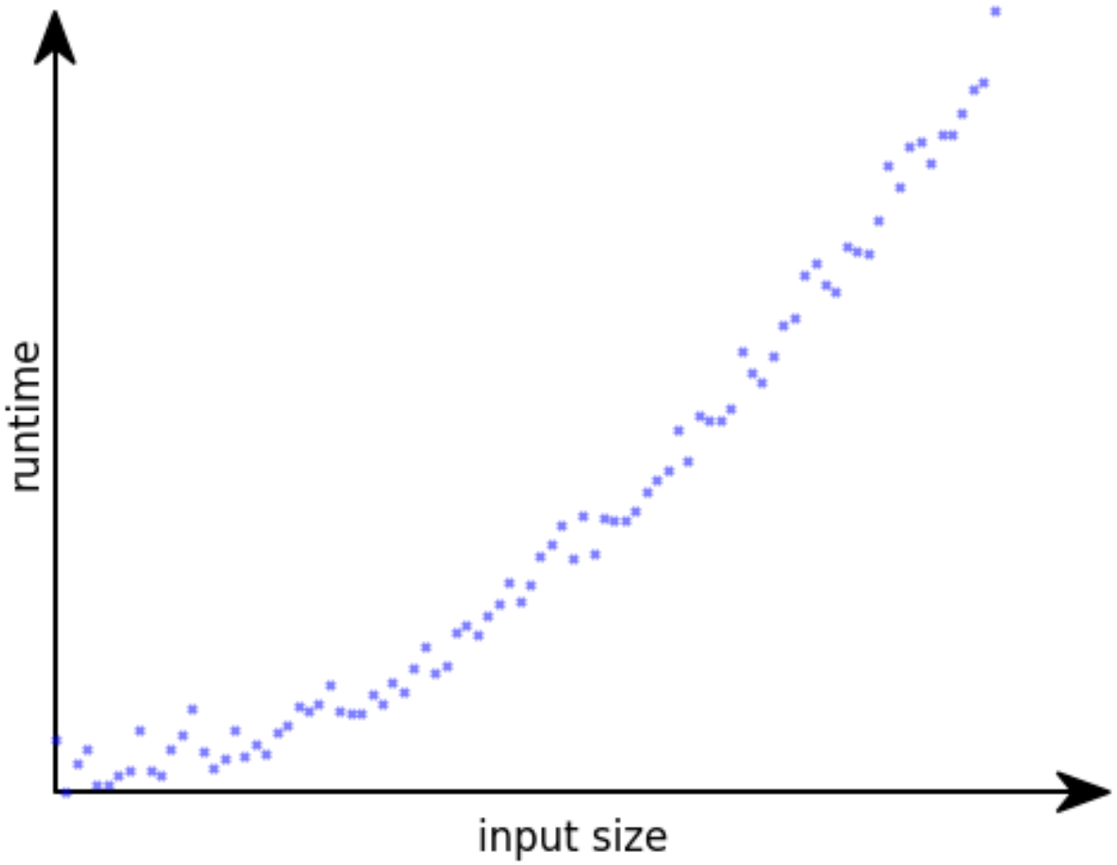
Idea:

Invariant: The beginning (prefix) of the array is sorted.

- Initially, only the first element is sorted, gradually more elements are added.
- Gradually insert each element in its correct position.

```
InsertionSort(array):
for i = 1 to n-1 do
    // Prefix A[0..i] is already sorted
    pos = i
    while (pos > 0) and (A[pos] < A[pos-1]) do
        swap(A[pos], A[pos-1])
        pos = pos - 1
```

Insertion Sort: Runtime Analysis



still looks like quadratic runtime

☹